

Server & Website Management Platform

Git & GitHub Team Development & Branching Workflow Standard

App: server_management **Platform:** Frappe Framework v16 **Developers:** Rohan & Roshni **Date:** September 2026

1. Branching Strategy & Architecture

To ensure clean collaboration without code loss or unreviewed production changes, our team follows a strict 4-branch model:

Branch Name	Access / Role	Description & Policy
main	Production Release	Protected Branch. Contains only thoroughly tested, production-ready code. Never push directly to main.
development	Integration & Staging	Shared Team Branch. Rohan and Roshni merge their tested feature work here via Pull Requests.
rohan	Developer Workspace	Private Feature Branch for Rohan. Rohan develops, tests, and pushes his daily commits here.
roshni	Developer Workspace	Private Feature Branch for Roshni. Roshni develops, tests, and pushes her daily commits here.

Golden Rule of Team Development:

Always do your work in your dedicated branch (rohan or roshni). Pull the latest updates from development before starting work, and submit Pull Requests on GitHub when your feature is complete.

2. Developer Daily Workflow Guide

Workflow A: Developer Rohan (Working on rohan branch)

Follow these steps whenever making any changes or additions to the codebase:

```
# Step 1: Navigate to the app directory
cd /home/frappe/frappe-bench-v16/apps/server_management

# Step 2: Switch to your branch and pull latest changes
git checkout rohan
git pull origin rohan

# Step 3: (Recommended) Sync with latest team updates from development
git pull origin development

# Step 4: Make your code edits in VS Code / IDE...

# Step 5: Check modified files and status
git status

# Step 6: Stage and commit with a standard message
git add .
git commit -m "feat: implement Server controller validations in server.py"

# Step 7: Push ONLY to your own branch on GitHub
git push origin rohan
```

📁 **Workflow B: Developer Roshni (Working on roshni branch)**

Follow these steps whenever making any changes or additions to the codebase:

```
# Step 1: Navigate to the app directory
cd /home/frappe/frappe-bench-v16/apps/server_management

# Step 2: Switch to your branch and pull latest changes
git checkout roshni
git pull origin roshni

# Step 3: (Recommended) Sync with latest team updates from development
git pull origin development

# Step 4: Make your code edits in VS Code / IDE...

# Step 5: Check modified files and status
git status

# Step 6: Stage and commit with a standard message
git add .
git commit -m "feat: add Website DocType schema and client script"

# Step 7: Push ONLY to your own branch on GitHub
git push origin roshni
```

3. Merging Changes into development (Pull Requests)

Once a module or feature is completed and tested on your branch, merge it into the shared development branch using GitHub Pull Requests:

Standard GitHub PR Workflow:

- Go to GitHub: https://github.com/rohansapkale/server_management
- Click **"Pull requests"** → **"New pull request"**
- Set **Base:** development ← **Compare:** rohan (or roshni)
- Review the green/red line diffs to verify your changes.
- Click **"Create pull request"**, add a summary of changes, and request code review.
- Once reviewed and approved, click **"Merge pull request"** → **"Confirm merge"**.

4. Conventional Commit Standards

To keep the project Git history clean and audit-ready, prefix all commit messages with standard tags:

Prefix	Purpose	Example Commit Message
feat:	New feature or DocType implementation	feat: add Domain expiry calculation logic
fix:	Bug fix or validation error patch	fix: prevent deployment to offline servers
refactor:	Code reorganization with no behavior change	refactor: simplify server status helper method
docs:	Documentation or README updates	docs: update developer onboarding instructions
test:	Adding or updating unit test cases	test: add automated test for backup expiry date

5. Conflict Resolution Runbook

If two developers edit the same line of code in the same file, Git will highlight a conflict during merge:

⚠ **Resolving Merge Conflicts:**

1. Open the conflicted file in your IDE. You will see markers:

```
<<<<<<< HEAD (current branch)
self.validate_server()
=====
self.validate_domain()
>>>>>>> roshni (incoming branch)
```

2. Keep both or edit to the correct combined logic:

```
self.validate_server()
self.validate_domain()
```

3. Save the file, stage, and complete the merge:

```
git add .
git commit -m "fix: resolve merge conflict in website.py"
git push origin development
```

6. Git Command Cheat Sheet & Safety Guide

Action / Task	Command
Check current branch and modified files	git status
List all local branches	git branch
Switch to another branch	git checkout <branch_name>
Pull latest commits from remote branch	git pull origin <branch_name>
Stage all modified files	git add .
Commit staged changes	git commit -m "feat: description"
Push commits to remote branch	git push origin <branch_name>
Discard uncommitted changes in a specific file	git checkout -- <file_path>
Temporarily stash uncommitted changes	git stash (restore later with git stash pop)
View recent commit log	git log --oneline -n 10